

DZIEŃ 3

MODUŁ 1 — PLIKI I FOLDERY

Zadanie 1 — Kopiowanie pliku

Folder: *Dzien3_Zadanie1_Pliki_Foldery/* · Temat: *Operacje na plikach + Path.Combine + Path.Exists*

OPIS ZADANIA

1. Pobierz bieżący katalog projektu (`Directory.GetCurrentDirectory`)
2. Użytkownik wybiera folder docelowy (`Select Folder`)
3. Iteruj po plikach `*.xlsx` w bieżącym folderze (`For Each File in Folder`)
4. Skopiuj plik do folderu docelowego (`Copy File` , Overwrite = True)
5. Sprawdź czy skopiowany plik istnieje (`Path.Exists`)
6. Zaloguj wynik: `"Czy skopiowano plik? " & bool_czy_skopiowano_plik`

WSKAZÓWKI

- `Directory.GetCurrentDirectory` — sposób pobrania katalogu roboczego procesu
- `Path.Combine(str_docelowy_folder, CurrentFile.Name)` — bezpieczne łączenie ścieżek
- `Path.Exists` — walidacja czy operacja się powiodła

ZMIENNE

Nazwa	Typ	Opis
str_aktualny_folder	String	Bieżący katalog projektu
str_docelowy_folder	String	Folder wybrany przez użytkownika
bool_czy_skopiowano_plik	Boolean	Wynik sprawdzenia Path.Exists

Zadanie 2 — Rozpakowanie ZIP i raport CSV

Folder: *Dzien3_Zadanie2_Pliki_Foldery/* · Temat: *Extract ZIP + Build Data Table + Write CSV*

OPIS ZADANIA

1. Rozpakuj plik `Raporty.zip` (`Extract/Unzip Files` , `ExtractToADedicatedFolder = True`)
2. Pobierz ścieżkę rozpakowanego folderu z `DestinationFolderInfo`
3. Stwórz tabelę: `Nazwa pliku` , `Folder` , `Rozszerzenie` , `Rozmiar` (`Build Data Table`)
4. Iteruj po rozpakowanych plikach (`For Each File in Folder`):
 - `Modify Text` — usuń rozszerzenie z nazwy
 - `Modify Text` — rozszerzenie bez kropki + `ToUpper`
 - `Add Data Row`
5. Zapisz tabelę do pliku CSV (`Write CSV`)

ZMIENNE

Nazwa	Typ	Opis
<code>dirinfo_folder_rozpakowane</code>	<code>DirectoryInfo</code>	Info o folderze docelowym
<code>str_folder_rozpakowane</code>	<code>String</code>	Pełna ścieżka folderu
<code>dt_detale_plikow</code>	<code>DataTable</code>	Tabela z detalami plików
<code>str_nazwa_pliku</code>	<code>String</code>	Nazwa pliku bez rozszerzenia
<code>str_rozszerzenie</code>	<code>String</code>	Rozszerzenie (uppercase, bez kropki)

Zadanie 3 — Sortowanie plików wg rozszerzenia

Folder: `Dzien3_Zadanie3_Pliki_Foldery/` · Temat: `Switch` + `Move File` + `Create Folder` + `Array`

OPIS ZADANIA

1. Zbuduj ścieżkę: `Path.Combine(Directory.GetCurrentDirectory, "Pliki do zadan")`
2. Stwórz tablicę folderów: `{"PDF", "DOCX", "XLSX"}`
3. W pętli `For Each` stwórz foldery (`Create Folder`)
4. W pętli `For Each File in Folder` :
 - `Switch<String>` na `CurrentFile.Extension.Replace(".",Nothing).ToUpper`
 - Case `PDF` → `Move File` do folderu PDF
 - Case `DOCX` → `Move File` do folderu DOCX
 - Case `XLSX` → `Move File` do folderu XLSX
 - Default → pusta sekwencja

ZMIENNE

Nazwa	Typ	Opis
<code>str_folder_z_plikami</code>	<code>String</code>	Ścieżka do folderu z plikami
<code>arr_foldery_docelowe</code>	<code>String[]</code>	Array z nazwami folderów docelowych
<code>ilr_stworzony_folder</code>	<code>ILocalResource</code>	Referencja do stworzonego folderu

MODUŁ 2 — FLOWCHART: BIAŁA LISTA (CEIDG)

Ćwiczenie 4 — Weryfikacja NIP na Białej Liście

Folder: `Dzien3_Zadanie4_Flowchart_CEIDG/` · Temat: `Flowchart` + `pętla` + `Check App State` + `DataTable`

OPIS ZADANIA

1. Sprawdź czy plik `CEIDG.xlsx` istnieje (`Path Exists`)
2. `FlowDecision` — plik istnieje → kontynuuj; nie → koniec
3. Wczytaj plik do `DataTable` (`Read Range`)
4. Otwórz stronę (`https://podatki-arch.mf.gov.pl/wykaz-podatnikow-vat-wyszukiwarka/`)
5. Pętla (strzałka powrotna w Flowchart):
 - Inkrementuj licznik
 - `FlowDecision` — `int_licznik > int_liczba_wierszy` → koniec
 - Wpisz NIP z tabeli: `row_wiersz_tabeli.Item("NIP").ToString`
 - Kliknij "Szukaj"
 - `Check App State` — sprawdź wynik weryfikacji
 - Zapisz wynik do odpowiedniej komórki w pliku excel: `"Figuruje na bialej liscie"/"Nie figuruje na bialej liscie"`
6. Po pętli zamknij stronę

Wskazówka: W Flowchart pętla jest bardziej widoczna niż w Sequence — strzałka powrotna do kroku z inkrementacją licznika tworzy pętlę.

ZMIENNE

Nazwa	Typ	Opis
<code>bl_czy_plik_istnieje</code>	Boolean	Czy plik CEIDG.xlsx istnieje
<code>dt_CEIDG</code>	DataTable	Dane z pliku CEIDG
<code>int_liczba_wierszy</code>	Int32	Liczba wierszy w tabeli
<code>int_licznik</code>	Int32	Bieżący numer wiersza
<code>dtrow_wiersz_tabeli</code>	DataRow	Bieżący wiersz tabeli
<code>str_nip</code>	String	Aktualnie przetwarzany NIP

MODUŁ 3 — ORCHESTRATOR (ASSETS, QUEUES, DISPATCHER/PERFORMER)

Zadanie 5 — Logowanie z Orchestratora

Folder: *Dzien3_Zadanie5_App_Logowanie/* · Temat: *Get Credential + SecureString + Pick/PickBranch*

OPIS ZADANIA

1. Pobierz login i hasło: `Get Credential` → Asset: `Robotivity_App_Credentials` , FolderPath: `Robotivity2`
2. Pobierz ścieżkę: `Get Asset` → Asset: `Sciezka_Aplikacji` , FolderPath: `Robotivity2`
3. Zamknij Robotivity (`Kill Process "robotivity"`)
4. Otwórz Robotivity (`Use Application/Browser`)
5. Wpisz login (`Type Into` → Text)
6. Wpisz hasło (`Type Into` → `SecureText!`)
7. Kliknij "LOG IN"
8. Użyj aktywności `Pick` z dwoma `PickBranch` :
 - **Branch 1 (Zalogowano):** Trigger: `Check App State` na przycisk "EXIT" → Log: "Zalogowano do programu"
 - **Branch 2 (Błędne hasło):** Trigger: `Check App State` na tekst "The User name or password you enter is incorrect" → Click "OK" → Log (Warn): "Nie zalogowano do aplikacji. Sprawdź hasło"

Pick/PickBranch — aktywność "wyścigowa": pierwsza gałąź, której trigger się spełni, wygrywa.

Używamy jej zamiast pojedynczego Check App State, bo mamy dwa możliwe scenariusze.

Uwaga: W Type Into dla hasła używaj właściwości `SecureText` (nie `Text` !) — hasło nigdy nie będzie widoczne w logach.

ZMIENNE

Nazwa	Typ	Opis
str_nazwa_uzytkownika	String	Login z Credential
sstr_haslo	SecureString	Hasło z Credential
str_sciezka_do_aplikacji	String	Ścieżka do Robotivity.exe

Zadanie 6 — Dispatcher

Folder: `Dzien3_Zadanie6_Dispatcher/` · Temat: `Bulk Add Queue Items + Add Queue Item z referencjami`

OPIS ZADANIA

- Wczytaj `Transactions.xlsx` w `Try Catch` :
 - Try:** `Read Range Workbook` (`AddHeaders = True`)
 - Catch:** `Kill Process "EXCEL"` → `Delay 2s` → ponowny `Read Range`

Sposób 1 — Bulk Add (hurtowy, zakomentowany w projekcie):

- `Bulk Add Queue Items` — `QueueName: Robotivity_Bulk` , `FolderPath: Robotivity2`

Sposób 2 — Add Queue Item z referencjami (aktywny):

- W pętli `For Each Row` :
 - `Add Queue Item` — `QueueName: Robotivity_Single` , `FolderPath: Robotivity2`
 - Reference: `CurrentRow.Item("CashIn").ToString & "_" & CurrentRow.Item("OnUsCheck").ToString`
 - ItemInformation: `CashIn, OnUsCheck, NotOnUsCheck`
- Log Message: "Transakcje dodane do kolejki"

Wskazówka: Try Catch wokół `Read Range` — Excel może blokować plik; w Catch: `Kill Process EXCEL + Delay + ponów`. `Bulk Add` jest szybszy, ale nie pozwala nadać indywidualnych referencji.

ZMIENNE

Nazwa	Typ	Opis
dt_transakcje	DataTable	Dane z Transactions.xlsx

Zadanie 7 — Performer

Folder: `Dzien3_Zadanie7_Performer/` · Temat: `Flowchart + Queue + Robotivity + obsługa błędów`

OPIS ZADANIA (FLOWCHART)

1. `Get Asset` — pobierz `Sciezka_Aplikacji` z folderu `Robotivity2`
2. `Invoke Workflow File` — `Robotivity_App_Logowanie.xaml` (argumenty: `str_sciezka_aplikacji` in, `out_bool_zalogowano` out)
3. `FlowDecision` — `out_bool_zalogowano` ?
 - False → "Koniec Pracy Robota" (If: zalogowano → log / Else → log "Nie zalogowano") + `Kill Process "robotivity"`
 - True → kontynuuj
4. Pętla (strzałka powrotna):
 - `Try Catch` : `Get Transaction Item` z kolejki `Robitivity_Single` (FolderPath: `Robotivity2`)
 - Catch: `qi_transaction_item = Nothing`
 - `FlowDecision` — `qi_transaction_item IsNot Nothing` ?
 - True → Log "Reference: " + Reference → `Invoke Workflow File` `Procesowanie_Transakcji.xaml`
 - W `Procesowanie_Transakcji.xaml` : wyciągnij dane, sprawdź `.IsNumeric` , Type Into + Click Accept lub Set Transaction Status = Failed

Wskazówka: Logowanie i procesowanie są wydzielone do osobnych plików .xaml (reużywalność). W Try Catch: gdy kolejka pusta → Exception → Catch: `qi_transaction_item = Nothing` → pętla się zakończy.

ZMIENNE

Nazwa	Typ	Opis
<code>out_bool_zalogowano</code>	Boolean	Czy logowanie się powiodło (z Invoke Workflow)
<code>str_sciezka_aplikacji</code>	String	Ścieżka do Robotivity (z Get Asset)
<code>qi_transaction_item</code>	QueueItem	Bieżąca transakcja z kolejki

BONUS Zadanie dodatkowe — Dashboard transakcji

Temat: Podsumowanie przetworzonych transakcji w raporcie Excel

OPIS ZADANIA

1. Na początku procesu: `int_successful = 0` , `int_failed_business = 0`
2. W Performerze po Set Transaction Status: zlicz sukcesy/porażki
3. Na końcu: `Build Data Table` → `Add Data Row` → `Write Range` do `Dashboard_transakcji.xlsx`

BONUS Zadanie dodatkowe — Monitoring folderu z raportem email

Temat: *For Each File in Folder + CreationTime + SMTP (łączenie Dnia 2 z Dniem 3)*

OPIS ZADANIA

1. Monitoruj folder "Raporty" — sprawdź pliki `*.xlsx` utworzone w ostatnich 24h
2. Użyj: `dat_granica = Now.AddHours(-24)`
3. Zbierz nazwy nowych plików do `List(Of String)`
4. Jeśli są nowe pliki — wyślij email z listą przez SMTP

Robotivity® — Autoryzowany Partner UiPath
info@robotivity.pl · www.Robotivity.pl